

How to Code With AI.

Build a guarded agent in 90 minutes.



I'm Kanis.

I turn complex physical systems into AI problems.

Computational simulation at IIT and the University of Alberta — fluid dynamics, numerical solvers, ML on physical phenomena. Production ML at PwC and AltaML. Then co-founded BinoLoop as CTO, building AI perception (vision + speech + LLMs) for regulated, high-stakes work.

Now at Founsi: point your camera, your inventory builds itself. Multimodal AI closing the gap between digital intelligence and what you actually own.

I'VE BUILT AT

IIT (ISM)

U Alberta

PwC

AltaML

BinoLoop

Founsi.

What you walk out with.

- ✓ A working Cursor setup
- ✓ A guarded agent you built yourself
- ✓ Your first AGENTS.md
- ✓ A mental model that scales to any AI tool
- ✓ A bookmarks page worth 6 months of self-study

Coding changed shape.

2024

Writing code.

Tab to autocomplete.
Copy from Stack Overflow.
Debug it yourself.
Ship it yourself.

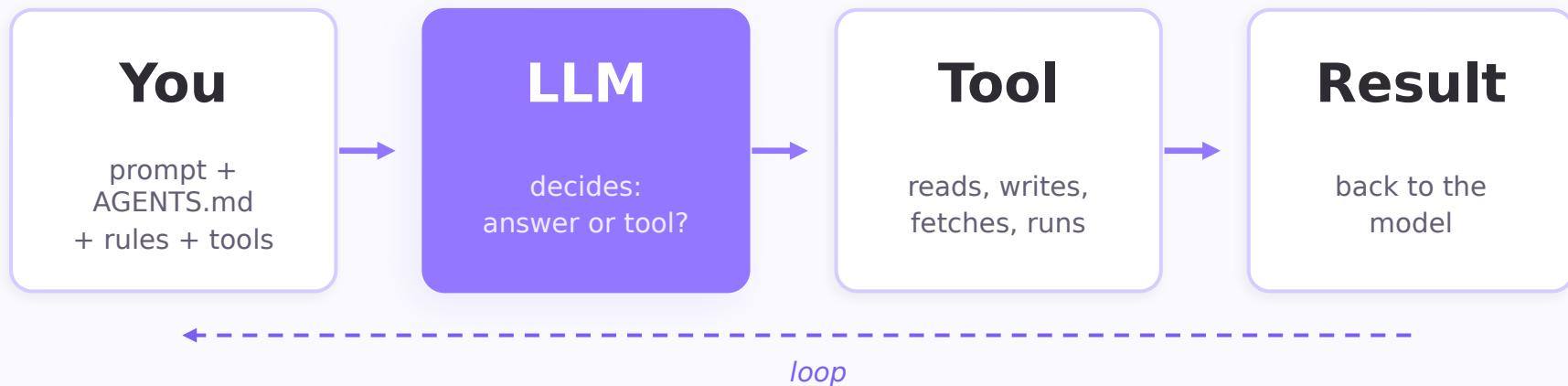
2026

Directing code.

Write a prompt.
Review a diff.
Approve or redline.
Keep moving.

Different skill. Tonight is about training the second one.

An agent is a loop.



Everything tonight is about getting the right stuff into that loop.

Everything fits into four buckets.

1

Context

AGENTS.md
.cursorrules

2

Rules & Memory

What it remembers,
what you tell it once.

3

Tools & MCP

What it can do.
What it can't.

4

Safeguards

How you don't
get burned.

Four minutes each. Open Cursor now if you haven't.

The Guarded Agent.

Your Guarded Agent

- Scoped to one folder
- Tool allowlist
- Approval before destructive
- No secrets in context
- Cost cap
- Audit log

Real engineers don't ship unguarded agents. Tonight you build with the rails on.



Open Cursor right now.

- 1 Cursor is open
- 2 Model = Claude Sonnet 4.6 (or whatever Cursor offers)
- 3 Create folder ~/sait-workshop and open it
- 4 Open the integrated terminal (Ctrl + `)

No API keys needed. Cursor's bundled inference is enough.

01

Context.

AGENTS.md · .cursorrules

*The single highest-leverage thing
you'll learn tonight.*

Meet AGENTS.md.

The README, but written for the AI. It's the universal context file — Cursor reads it. Claude Code reads it. Codex reads it.

One file. Every tool. You write it once, every agent that touches your repo gets the same briefing.

spec → agents.md

What goes in.

1. What this project is

One paragraph. Plain English.

3. How to run / test

Exact commands. Copy-paste runnable.

5. Do / Don't

The agent's guardrails.

2. Tech stack

Languages, frameworks, key libs.

4. Conventions

Naming, structure, the invisible rules.

6. File map

Where things live.

→ *examples in the wild*:

[agents.md](#)

github.com/sst/opencode

[appendix → A1](#)



Write your AGENTS.md.

I'll write one on screen. You write one in your folder.

```
# AGENTS.md
```

```
## 1. What this project is
```

```
A CLI that summarizes PDFs in a folder.
```

```
Built for me. Pre-MVP. Just runs locally.
```

```
## 6. Guardrails
```

- Never run `rm`, `git push --force...`
- Never read `.env` or `secrets/`
- Explain before non-trivial edits

Save. Then ask Cursor a question. Watch it cite the file.

02

Rules & Memory.

Operating standards. What it remembers.

AGENTS.md is the briefing. Rules are the standing orders.

Four layers, broadest to most specific.

Global rules

preferences across all projects

Project rules

.cursorrules / .cursor/rules/*.md

AGENTS.md

this project's briefing

Memory

facts it learned in conversation

→ *inspiration:*

docs.cursor.com/rules

cursor.directory

[awesome-cursorrules](#)

What goes in rules. What doesn't.

✓ DO put in rules

- "Always run tests after editing"
- "Prefer pytest over unittest"
- "Don't touch migrations without asking"
- "Explain before non-trivial edits"

✗ DON'T

- Details that change daily
- The whole architecture
- Secrets, tokens, keys
- Anything the AI can read from the code

Rules are for the invisible — preferences, priorities, prior incidents.

Rules look different by stack.

Backend

- Run pytest after every src/ change
- Show migration SQL before applying
- Type hints required; mypy clean

Frontend

- Use design system tokens; no custom colors
- Aria labels on every interactive element
- Verify mobile-first at 380px

DevOps / SRE

- terraform plan before any apply
- Audit log every infra change
- Dry-run all destructive commands

Data

- Never log row contents; PII flagged
- Parameterized queries only
- Verify counts before & after migrations

Same skeleton, different priorities. Full templates in appendix.



Add one rule.

Create `.cursorrules` in your folder. Add one rule that matters. Pick from below — or invent one.

`.cursorrules`

- Always explain edits before making them.
- After any change to `src/`, run `pytest` and show me the output.
- Never run destructive shell commands without asking.
- Keep diffs small; one concern per turn.

03

Tools & MCP.

What the agent can do.

Context is knowledge. Tools are capability.

MCP servers expose three things.

Model Context Protocol — Anthropic's standard for plugging external capabilities into AI clients. Cursor, Claude Code, Codex all speak it. An MCP server can expose any combination of:

Tools

Functions the model can call.
(`read_file`, `run_query`,
`send_message`)

Resources

Read-only data the model can
fetch.
(a doc, a config, a record set)

Prompts

Templated workflows the user
can trigger.
("summarize_pr", "file_bug")

Sequential-thinking? It's a reasoning aid. A Prompt, not a Tool.

The four to install first.



filesystem

TOOL

Read & write files. Scope-limited to one folder.



fetch

TOOL

Pull URLs into context.



git

TOOL

Work with branches & commits.



sequential-thinking

REASONING

Structured scratchpad for hard problems.

Tools do things. Reasoning servers help the model think. Both useful — for different reasons.

Where to find more.

MCP MARKETPLACES

mcp.so

searchable directory of servers

smithery.ai

registry + one-click install

github.com/modelcontextprotocol

official reference servers

awesome-mcp-servers

community-curated list

LIVE DEMO COMING UP

Figma MCP

"Turn this Figma frame into a React component." Cursor pulls the real layers, writes the code.

Step-by-step in appendix → A7

FREE FULL-APP DESIGNS TO CLONE & CONVERT

AI Chat Apps

SaaS Dashboards

Fintech Apps

Landing Pages

→ [figma-experiments.md](#)

Filesystem MCP, scoped.

The path you pass to the server is the wall. Outside it, the agent literally cannot read or write — not asking nicely, structurally blocked.

```
{
  "mcpServers": {
    "filesystem": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-filesystem",
               "/Users/you/sait-workshop"]
    }
  }
}
```

04

Safeguards.

How you don't get burned.

*The part nobody talks about
until they've been burned.*

Five guards. Always.

1 Don't auto-run commands you can't read

Cursor will offer Auto-Run. Off until you trust the project.

2 Commit before letting the agent loose

A clean git status is your undo button.

3 .env discipline

No real secrets where the agent's context can reach.

4 Read every diff

Especially the long ones.

5 Scope your asks

"Refactor the whole thing" is how bad PRs are born.

Run the whole thing offline.

Ollama + LM Studio let you run open models — Llama, Qwen, Mistral — on your laptop. The most guarded agent in the world is the one that never sends data anywhere.

	Cloud (Claude/GPT)	Local (Ollama)
Quality	★★★★★	★★★ (improving)
Speed	Fast	Depends on hardware
Cost	API \$\$	Free
Privacy	Sent to provider	Stays on machine
Offline	No	Yes

Bonus Lab 5: swap the brain to Ollama. WiFi off. Agent still works.

TAKE A PHOTO

Bookmark these.

AWESOME LISTS

github.com/PatrickJS/awesome-cursorrules
github.com/punkpeye/awesome-mcp-servers
github.com/asgeirtj/system_prompts_leaks

CURSOR & TOOLS

cursor.directory
docs.cursor.com
modelcontextprotocol.io

PROMPT LIBRARIES

docs.anthropic.com/claude/prompt-library
cookbook.openai.com
promptingguide.ai

LOCAL MODELS

ollama.com
lmstudio.ai
huggingface.co/models

Full lists in the repo: github.com/remembr-xyz/code-with-ai

NEXT 90 MINUTES

Six labs. One agent.

6:30	00	Setup	Cursor + project folder
6:40	01	AGENTS.md	Real context file for your project
7:00	02	MCP, scoped	Filesystem MCP locked to your folder
7:20	03	Cursor is the agent	Make the loop visible; tighten one rule
7:40	04	Red team	Break your own guardrails, then patch
7:55	05	Local models (bonus)	Cursor → Ollama → WiFi off



The memory layer for your physical life.

Founsi catalogs, tracks, and remembers everything you own — so you don't have to.



Raising pre-seed

If you build with AI, let's talk.



Open beta soon

Scan the next slide to join the waitlist.



Design partners

If you've ever lost something expensive, find me after.

Subscribe. Clone. Find me.

Three QR codes. Take a photo before you leave.

[QR]

Newsletter

learnagentic.substack.com

[QR]

Workshop repo

[github.com/remembr-xyz/
code-with-ai](https://github.com/remembr-xyz/code-with-ai)

[QR]

Find me on X

[@above_almighty](https://twitter.com/above_almighty)

APPENDIX

Reference material.

Examples in the wild, full rule templates by role, MCP type breakdown, step-by-step setup guides for Figma + Stitch + filesystem MCP, lab quick-reference. Use this during the hands-on or take it home.

AGENTS.md in the wild.

Read three of these end-to-end. You'll learn faster than from any tutorial.

agents.md	The official spec + canonical examples.
github.com/sst/opencode	Excellent AGENTS.md for a real open-source agent.
github.com/openai/codex	OpenAI's coding agent — sets the standard for tool authors.
github.com/elysiajs/elysia	Web-framework example. Shows tech-stack conventions.
github.com/cline/cline	Comprehensive rules-style AGENTS.md.
github.com/vercel/ai	AI SDK — strong file-map section worth copying.

AGENTS.md is still evolving. The spec stabilized in late 2024; tools now compete on how well they read it.

Cursor rules: where to crib from.

Start with the official docs, then plunder `cursor.directory` for your stack.

OFFICIAL	<code>docs.cursor.com/context/rules</code>	How Cursor reads rules. Read this once.
OFFICIAL	<code>github.com/getcursor/cursor</code>	Cursor's own repo — has reference rules examples.
COMMUNITY	<code>cursor.directory</code>	Searchable directory. Filter by stack (React, FastAPI, Rust, Go, etc.).
COMMUNITY	<code>github.com/PatrickJS/awesome-cursorrules</code>	The biggest curated rules library.
DEEP CUT	<code>github.com/asgeirtj/system_prompts_leaks</code>	Leaked system prompts of major tools — gold for learning what good prompting looks like.

Rules template: Backend.

Drop into .cursorrules. Tune the specifics to your stack and team.

Operating rules – backend

Testing

- Run pytest after every change in src/. Show output.
- New public function gets at least one test.

Safety

- Never run migrations without showing me the SQL first.
- Never commit .env, *.key, anything in secrets/.
- Use parameterized queries; never string-concat SQL.

Style

- Type hints on every public function. mypy clean before commit.
- Errors: raise specific exceptions; never bare except.
- Imports: absolute from src/; never wildcard.

Rules template: Frontend.

Design system + accessibility + small components. Adjust for React/Vue/Svelte specifics.

Operating rules – frontend

Design system

- Use design tokens; never invent hex codes inline.
- Match existing spacing scale (4, 8, 16, 24, 32, 48).
- One Tailwind class per concern; no arbitrary [w-[342px]].

Accessibility

- Aria labels on every interactive element (button, input, link).
- Focus indicators visible; 2px outline minimum.
- Verify mobile-first at 380px; then scale up.

Components

- Prefer composition over inheritance. Small components win.
- Props typed with TypeScript; never any without a TODO.
- New component? Add it to Storybook with at least 3 states.

Rules template: DevOps & Data.

DevOps / SRE

- terraform plan before any apply.
Show me the diff. I approve.
- Never kubectl delete without confirm.
- Audit log every infra change.
- Dry-run all destructive commands.
- Treat prod like prod, not staging.
- Secrets in Vault/SOPS, never .env in prod.
- New service? Add SLOs in the same PR.

Data engineering

- PII columns flagged in schema.
Never log row contents.
- Parameterized queries only. No concat.
- Schema changes require RFC PR first.
- Verify row counts before/after migration.
- Backfills run in batches; checkpointed.
- Lineage: every table has owner + source.
- Sample query results, never SELECT *.

Adding an MCP server in Cursor.

- 1 Open MCP settings** Cursor → Settings (⌘,) → search "MCP" → click "MCP Servers"
- 2 Edit mcp.json** Cursor opens the config file. Add a server entry under "mcpServers"
- 3 Specify command + args** command = how to run it (npx, python, node). args = arguments incl. scope path
- 4 Save → Cursor auto-starts** A green dot in the MCP list means the server is running
- 5 Test in chat** Open a new chat. The agent now has the server's tools available

Restart Cursor if a server fails to start. Most failures are missing npx / wrong path.

Figma MCP — full setup.

Lets your agent read your Figma frames and turn them into code. ~3 minutes to set up.

1. Get your Figma personal access token

Figma → Settings → Account → Personal access tokens → Generate. File content read.

2. Add to mcp.json

```
"figma": {  
  "command": "npx",  
  "args": ["-y", "figma-developer-mcp", "--stdio"],  
  "env": { "FIGMA_API_KEY": "figd_..." }}
```

3. Try it

Open chat: "Using Figma, fetch frame [URL] and write a React component matching the layout."

Same pattern: Figma Dev Mode MCP, figma-mcp, official Figma server — all work.

Other connectors worth trying.

Stitch (Google)

Google's design tool, similar to Figma. MCP exposes design boards as resources.

Search [smithery.ai](#) for "stitch" – install via Smithery's one-click flow.

Notion

Read/write Notion pages and databases. Great for spec-driven coding from a Notion brief.

`npm i @notionhq/notion-mcp-server` · needs Notion integration token

Postgres / SQLite

Read-only SQL from the agent. Use a dedicated read-only DB user.

`@modelcontextprotocol/server-postgres` or `server-sqlite`

GitHub

Issues, PRs, releases. Token-scoped to a single repo when possible.

`@modelcontextprotocol/server-github` · needs GitHub PAT

Lab quick-reference.

Full step-by-step is in the repo at github.com/remembr-xyz/code-with-ai/labs.

00	Setup	Cursor open · model picked · project folder ready
01	Write AGENTS.md	Customize 9-section template for your project; verify Cursor cites it
02	Install scoped filesystem MCP	Edit mcp.json with absolute path; verify out-of-scope read is blocked
03	Cursor IS your guarded agent	Give it a real task; watch the loop; add one new rule based on what you saw
04	Red-team	Path traversal · symlink · prompt injection · secret extraction · authority spoof
05	Local models (bonus)	Install Ollama · point Cursor at localhost:11434/v1 · WiFi off · agent still works

If you fall behind: skip ahead at the next lab boundary. Each is standalone.